

# **PROBLEM SOLVING AND TESTING**

NAME : ANUBHAV

VTU: 28767

COURSE CODE:10211CS227

WEEK 1 PROBLEMS

# SESSION 1

## 1.1-Java Lambda Expressions - <https://www.hackerrank.com/challenges/java-lambda-expressions/problem>

Test case 0

Test case 1

Test case 2

1 5

2 1 4

3 2 5

4 3 898

5 1 3

6 2 12

Expected Output

1 EVEN

2 PRIME

3 PALINDROME

4 ODD

5 COMPOSITE

Download

### CODE:

```
import java.io.*;
import java.util.*;

interface PerformOperation {
    boolean check(int a);
}

class MyMath {
    public static boolean checker(PerformOperation p, int num) {
        return p.check(num);
    }

    public PerformOperation isOdd() {
        return n -> n % 2 != 0;
    }
}
```

```

public PerformOperation isPrime() {
    return n -> {
        if (n < 2) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    };
}

```

```

public PerformOperation isPalindrome() {
    return n -> {
        String s = Integer.toString(n);
        String reversed = new StringBuilder(s).reverse().toString();
        return s.equals(reversed);
    };
}
} // Closes the MyMath class

```

```

public class Solution {

    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine());
        PerformOperation op;
        boolean ret = false;
        String ans = null;
    }
}

```

```

while (T--> 0) {
    String s = br.readLine().trim();
    StringTokenizer st = new StringTokenizer(s);
    int ch = Integer.parseInt(st.nextToken());
    int num = Integer.parseInt(st.nextToken());
    if (ch == 1) {
        op = ob.isOdd();
        ret = ob.checker(op, num);
        ans = (ret) ? "ODD" : "EVEN";
    } else if (ch == 2) {
        op = ob.isPrime();
        ret = ob.checker(op, num);
        ans = (ret) ? "PRIME" : "COMPOSITE";
    } else if (ch == 3) {
        op = ob.isPalindrome();
        ret = ob.checker(op, num);
        ans = (ret) ? "PALINDROME" : "NOT PALINDROME";
    }
    System.out.println(ans);
}
}
}
}

```

## SESSION 2

**2.1 - Java Sort – <https://www.hackerrank.com/challenges/java-sort/problem>**

## Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

✓ Test case 0

✓ Test case 1 

✓ Test case 2 

✓ Test case 3 

✓ Test case 4 

✓ Test case 5 

✓ Test case 6 

```
1 5
2 33 Rumpa 3.68
3 85 Ashis 3.85
4 56 Samiha 3.75
5 19 Samara 3.75
6 22 Fahim 3.76
```

Expected Output

```
1 Ashis
2 Fahim
3 Samara
4 Samiha
5 Rumpa
```

[Download](#)

## CODE:

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

class Student {
    private int id;
    private String firstName;
    private double cgpa;

    public Student(int id, String firstName, double cgpa) {
        this.id = id;
        this.firstName = firstName;
        this.cgpa = cgpa;
    }
}
```

```
public int getId() {  
    return id;  
}
```

```
public String getFirstName() {  
    return firstName;  
}
```

```
public double getCgpa() {  
    return cgpa;  
}  
}
```

```
public class Solution {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
        int testCases = Integer.parseInt(in.nextLine());  
  
        List<Student> studentList = new ArrayList<Student>();  
        while (testCases > 0) {  
            int id = in.nextInt();  
            String firstName = in.next();  
            double cgpa = in.nextDouble();  
  
            Student st = new Student(id, firstName, cgpa);  
            studentList.add(st);  
  
            testCases--;
```

```

    }

    in.close();

    // Java 8 elegant sorting using Comparator chaining
    studentList.sort(
        Comparator.comparing(Student::getCgpa).reversed()
            .thenComparing(Student::getFirstName)
            .thenComparing(Student::getId)
    );

    // Printing names using Java 8 Stream API
    studentList.stream()
        .map(Student::getFirstName)
        .forEach(System.out::println);
    }
}

```

## 2.2-. Sort Array By Parity <https://leetcode.com/problems/sort-array-by-parity/>

Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

[3, 1, 2, 4]

Output

[4, 2, 1, 3]

Expected

[2, 4, 3, 1]

Contribute a testcase

**CODE:**

```
class Solution {  
    public int[] sortArrayByParity(int[] nums) {  
        int left = 0;  
        int right = nums.length - 1;  
  
        while (left < right) {  
            // Find an odd number from the left  
            if (nums[left] % 2 == 0) {  
                left++;  
            }  
            // Find an even number from the right  
            else if (nums[right] % 2 != 0) {  
                right--;  
            }  
            // Swap if left is odd and right is even  
            else {  
                int temp = nums[left];  
                nums[left] = nums[right];  
                nums[right] = temp;  
                left++;  
                right--;  
            }  
        }  
        return nums;  
    }  
}
```

## SESSION 3

### 3.1-. Sorting: Comparator – <https://www.hackerrank.com/challenges/ctci-comparator-sorting/problem>

#### CODE:

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            // Sort descending by score
            return Integer.compare(b.score, a.score);
        }
        // If scores are equal, sort ascending by name alphabetically
        return a.name.compareTo(b.name);
    }
}
```

```
}  
}
```

```
public class Solution {  
    public static void main(String[] args) {  
        Scanner scan = new Scanner(System.in);  
        int n = scan.nextInt();  
  
        Player[] player = new Player[n];  
        Checker checker = new Checker();  
  
        for(int i = 0; i < n; i++){  
            player[i] = new Player(scan.next(), scan.nextInt());  
        }  
        scan.close();  
  
        Arrays.sort(player, checker);  
        for(int i = 0; i < player.length; i++){  
            System.out.printf("%s %s\n", player[i].name, player[i].score);  
        }  
    }  
}
```

The screenshot shows a coding challenge interface. On the left, a list of test cases from 0 to 7, all marked as passed with green checkmarks. Test case 0 is selected. The main area displays the input and expected output for Test case 0. The input consists of an integer 5 followed by five lines of player names and scores. The expected output lists the same players sorted by score in descending order.

| Test Case   | Status | Input (stdin)                                                        | Expected Output                                   |
|-------------|--------|----------------------------------------------------------------------|---------------------------------------------------|
| Test case 0 | Passed | 5<br>amy 100<br>david 100<br>heraldo 50<br>aakansha 75<br>aleksa 150 | aleksa 150<br>amy 100<br>david 100<br>aakansha 75 |
| Test case 1 | Passed |                                                                      |                                                   |
| Test case 2 | Passed |                                                                      |                                                   |
| Test case 3 | Passed |                                                                      |                                                   |
| Test case 4 | Passed |                                                                      |                                                   |
| Test case 5 | Passed |                                                                      |                                                   |
| Test case 6 | Passed |                                                                      |                                                   |
| Test case 7 | Passed |                                                                      |                                                   |

## 3.2-.Java Comparator – <https://www.hackerrank.com/challenges/java-comparator/problem>

The screenshot shows a green banner with the text "Congratulations" and "You solved this challenge. Would you like to challenge your friends?" followed by social media icons for Facebook, Twitter, and LinkedIn. A "Next Challenge" button is on the right. Below the banner, the test cases and input/output for Test case 0 are shown, identical to the first screenshot. A "Download" link is visible on the right side of the output area.

**Congratulations**

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#) [Next Challenge](#)

| Test Case   | Status | Input (stdin)                                                        | Expected Output                                                 |
|-------------|--------|----------------------------------------------------------------------|-----------------------------------------------------------------|
| Test case 0 | Passed | 5<br>amy 100<br>david 100<br>heraldo 50<br>aakansha 75<br>aleksa 150 | aleksa 150<br>amy 100<br>david 100<br>aakansha 75<br>heraldo 50 |
| Test case 1 | Passed |                                                                      |                                                                 |
| Test case 2 | Passed |                                                                      |                                                                 |
| Test case 3 | Passed |                                                                      |                                                                 |
| Test case 4 | Passed |                                                                      |                                                                 |
| Test case 5 | Passed |                                                                      |                                                                 |

## CODE:

```
import java.util.*;  
  
class Checker implements Comparator<Player> {
```

```

@Override

public int compare(Player a, Player b) {

    return Comparator.comparingInt((Player p) -> p.score)

        .reversed()

        .thenComparing(p -> p.name)

        .compare(a, b);

}

}

class Player{

    String name;

    int score;


    Player(String name, int score){

        this.name = name;

        this.score = score;

    }

}

```

```

class Solution {

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);

        int n = scan.nextInt();


        Player[] player = new Player[n];

        Checker checker = new Checker();


        for(int i = 0; i < n; i++){

            player[i] = new Player(scan.next(), scan.nextInt());

```

```

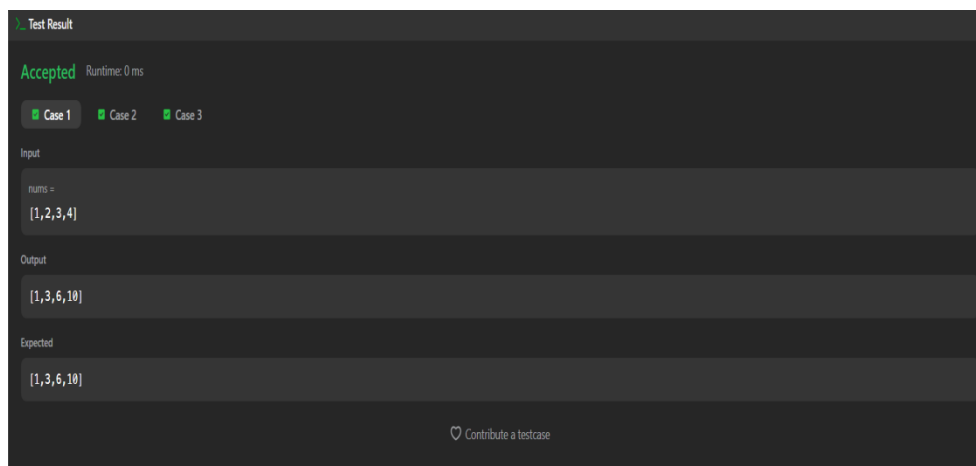
    }
    scan.close();

    Arrays.sort(player, checker);
    for(int i = 0; i < player.length; i++){
        System.out.printf("%s %s\n", player[i].name, player[i].score);
    }
}
}
}

```

## SESSION 4

### 4.1- Running Sum of 1D Array – <https://leetcode.com/problems/running-sum-of-1d-array/>



### CODE:

```

import java.util.Arrays;

public class Solution {

    public int[] runningSum(int[] nums) {
        // Start from index 1 and accumulate the sum in-place
        for (int i = 1; i < nums.length; i++) {

```

```

        nums[i] += nums[i - 1];
    }
    return nums;
}

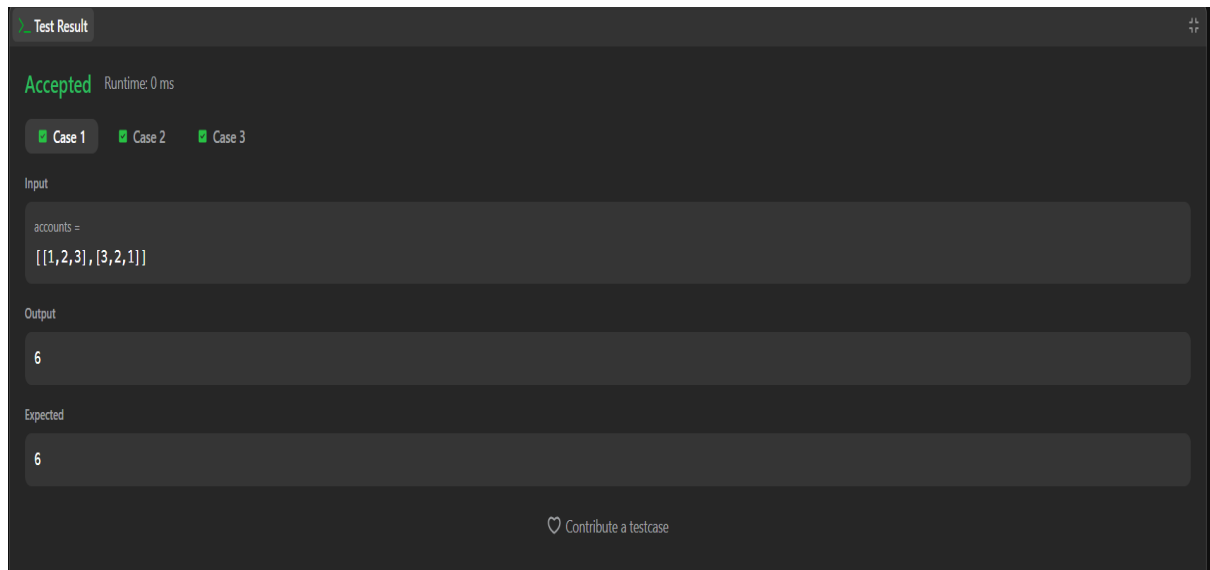
public static void main(String[] args) {
    Solution solver = new Solution();

    // Test Case 1
    int[] nums1 = {1, 2, 3, 4};
    System.out.println("Input: " + Arrays.toString(nums1));
    int[] result1 = solver.runningSum(nums1);
    System.out.println("Output: " + Arrays.toString(result1)); // Expected: [1, 3, 6,
10]
    System.out.println();

    // Test Case 2
    int[] nums2 = {3, 1, 2, 10, 1};
    System.out.println("Input: " + Arrays.toString(nums2));
    int[] result2 = solver.runningSum(nums2);
    System.out.println("Output: " + Arrays.toString(result2)); // Expected: [3, 4, 6,
16, 17]
}
}

```

## 4.2- Richest Customer Wealth – <https://leetcode.com/problems/richest-customer-wealth/>



## CODE:

```
import java.util.Arrays;
```

```
public class Solution {
```

```
    public int maximumWealth(int[][] accounts) {
```

```
        int maxWealth = 0;
```

```
        // Iterate through each customer
```

```
        for (int i = 0; i < accounts.length; i++) {
```

```
            int currentCustomerWealth = 0;
```

```
            // Sum up all bank accounts for the current customer
```

```
            for (int j = 0; j < accounts[i].length; j++) {
```

```
                currentCustomerWealth += accounts[i][j];
```

```
            }
```

```
            // Update maxWealth if the current customer is richer
```

```
            if (currentCustomerWealth > maxWealth) {
```

```
        maxWealth = currentCustomerWealth;
    }
}

return maxWealth;
}
```

```
public static void main(String[] args) {
    Solution solver = new Solution();

    // Test Case 1
    int[][] accounts1 = {
        {1, 2, 3},
        {3, 2, 1}
    };

    System.out.println("Output 1: " + solver.maximumWealth(accounts1)); //
Expected: 6
```

```
    // Test Case 2
    int[][] accounts2 = {
        {1, 5},
        {7, 3},
        {3, 5}
    };

    System.out.println("Output 2: " + solver.maximumWealth(accounts2)); //
Expected: 10
```

```
    // Test Case 3
    int[][] accounts3 = {
```

```

        {2, 8, 7},
        {7, 1, 3},
        {1, 9, 5}
    };

    System.out.println("Output 3: " + solver.maximumWealth(accounts3)); //
Expected: 17
    }
}

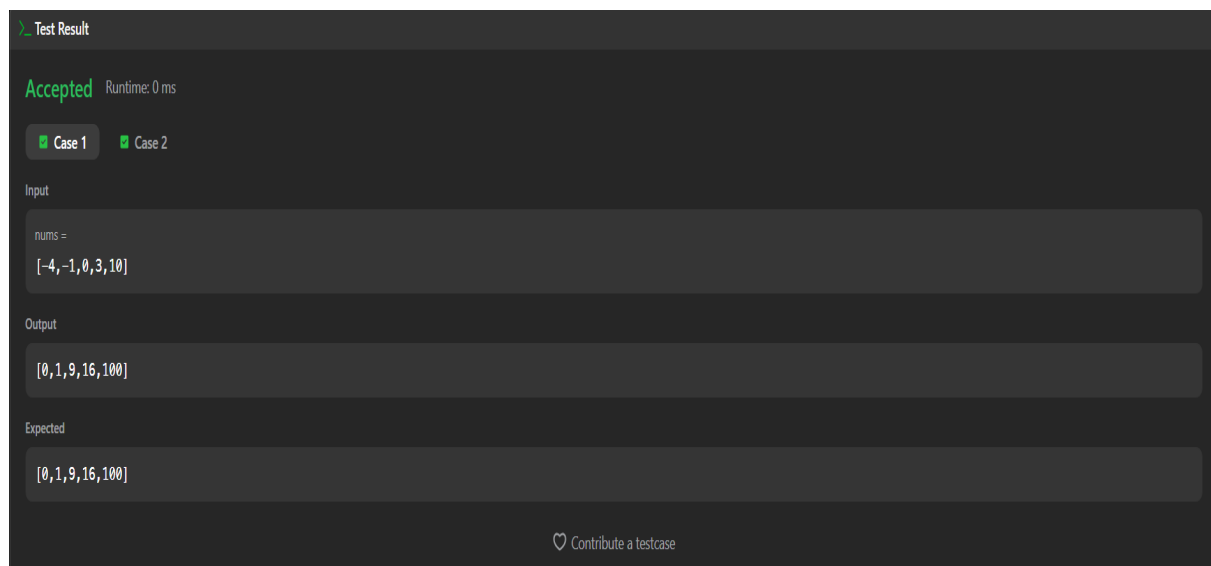
```

## SESSION 5

**5.1-Squares of a Sorted Array – <https://leetcode.com/problems/squares-of-a-sorted-array/>**

## SESSION 5

**5.1-Squares of a Sorted Array – <https://leetcode.com/problems/squares-of-a-sorted-array/>**



## CODE:

```
import java.util.Arrays;
```

```
public class Solution {

    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int left = 0;
        int right = n - 1;
        int index = n - 1; // Start filling the result array from the end

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            // Put the larger square at the current end position
            if (leftSquare > rightSquare) {
                result[index] = leftSquare;
                left++;
            } else {
                result[index] = rightSquare;
                right--;
            }
            index--;
        }

        return result;
    }

    public static void main(String[] args) {
```

```

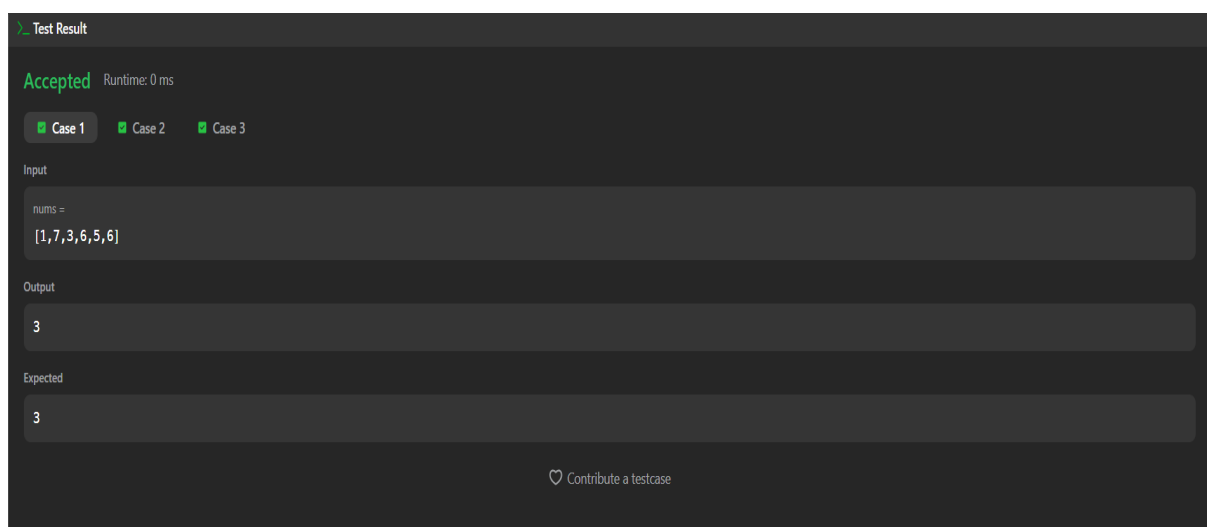
Solution solver = new Solution();

// Test Case 1
int[] nums1 = {-4, -1, 0, 3, 10};
System.out.println("Input 1: " + Arrays.toString(nums1));
System.out.println("Output 1: " +
Arrays.toString(solver.sortedSquares(nums1)));
// Expected: [0, 1, 9, 16, 100]
System.out.println();

// Test Case 2
int[] nums2 = {-7, -3, 2, 3, 11};
System.out.println("Input 2: " + Arrays.toString(nums2));
System.out.println("Output 2: " +
Arrays.toString(solver.sortedSquares(nums2)));
// Expected: [4, 9, 9, 49, 121]
}
}

```

## 5.2-Find Pivot Index – <https://leetcode.com/problems/find-pivot-index/>



## CODE:

```
import java.util.Arrays;
```

```
public class Solution {
```

```
    public int pivotIndex(int[] nums) {
```

```
        // Calculate the total sum of all elements in the array
```

```
        int totalSum = 0;
```

```
        for (int num : nums) {
```

```
            totalSum += num;
```

```
        }
```

```
        int leftSum = 0;
```

```
        // Iterate through the array to find the leftmost pivot index
```

```
        for (int i = 0; i < nums.length; i++) {
```

```
            // Right sum is totalSum minus leftSum minus the current element
```

```
            if (leftSum == totalSum - leftSum - nums[i]) {
```

```
                return i;
```

```
            }
```

```
            // Update leftSum for the next index position
```

```
            leftSum += nums[i];
```

```
        }
```

```
        // Return -1 if no pivot index satisfies the condition
```

```
        return -1;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
Solution solver = new Solution();
```

```
// Test Case 1
```

```
int[] nums1 = {1, 7, 3, 6, 5, 6};
```

```
System.out.println("Input 1: " + Arrays.toString(nums1));
```

```
System.out.println("Output 1: " + solver.pivotIndex(nums1)); // Expected: 3
```

```
System.out.println();
```

```
// Test Case 2
```

```
int[] nums2 = {1, 2, 3};
```

```
System.out.println("Input 2: " + Arrays.toString(nums2));
```

```
System.out.println("Output 2: " + solver.pivotIndex(nums2)); // Expected: -1
```

```
System.out.println();
```

```
// Test Case 3
```

```
int[] nums3 = {2, 1, -1};
```

```
System.out.println("Input 3: " + Arrays.toString(nums3));
```

```
System.out.println("Output 3: " + solver.pivotIndex(nums3)); // Expected: 0
```

```
}
```

```
}
```